微光招新题解

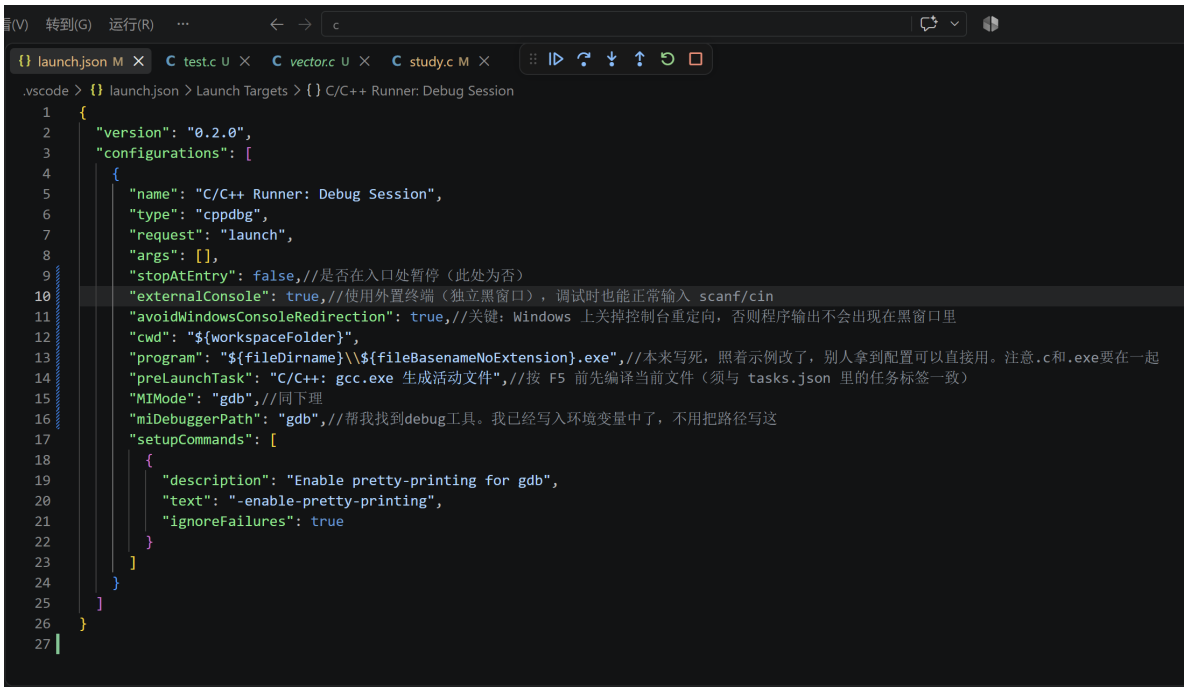
task 1

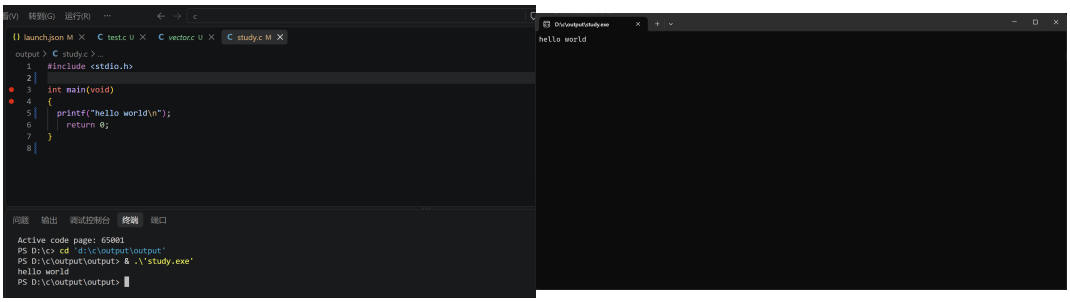
part-1：了解配置文件

1.gcc是早先用于Unix（Unix-like）系统的编译器，先面向c语言（GNU C Compiler），后适用于各种语言（GNU Compiler Collection）。MinGW是gcc面向Windows系统的移植版。

2.**task.json**代我执行编译命令，我可以在图形界面生成可执行文件。**launch.json**则用于运行和调试。**c_cpp_properties.json**用来包含头文件路径，用来理解代码（*include*，*宏*...）

3.插件其实不是必备的，但它能辅助开发，提高效率，如高亮，代码格式化，错误提醒





！！！此处让**ai大人**帮忙了：*"preLaunchTask": "C/C++: gcc.exe 生成活动文件"* 以及 *"avoidWindowsConsoleRedirection": true* ： 这个重定向才真正把I/O落点从vscode底部搬到独立窗口。true/false是有了独立窗口后才决定是否使用。

part-2：c语言基础

1.变量类型可以说是一种数据种类。**重要性：**【1】它们占据了不同大小内存空间，因此可以通过它们充分管理进而利用好内存。（但由于现在硬件进步，可能对此要求降低，比如python已经不再重视）。【2】变量类型的使用也涉及数据处理问题：好处在于分工明确，可控性强。坏处则是我们常常碰到数据截断等问题。【3】整数【4】char*（只可读）or char[] (可写)。

```
char c = 'a';               // 只能存一个字符
char s[6] = "apple";        // 用字符数组才能装下整个单词
printf("%c  %s\n", c, s);
```

2.数组下标通常从0开始。访问越界有诸多问题：**越界读取**（例如由于'0'被覆盖了，一直往后打印）应该会访问到垃圾数据或导致数据泄露（绕过canary，出现指针信息？有安全风险。或某些特定函数地址）。**越界写入**可能直接导致崩溃（落到未映射区——保护你的，发生段错误）或篡改数据（栈中的返回地址等又或者堆内存前后的管理元数据）。可能因为常忘记检查边界，循环时就出去了。还有字符 串处理时用数组也常常写出去。而危险性前面也提及了，程序崩溃还好，主要容易成为被攻击的目标。

3.for(初始化;条件;迭代)：初始化只做一次，条件要每轮先查才进循环，迭代在每轮循环结束后更新。while是先判断再执行，可能一次不跑，而do...while先执行一次再判断，所以至少跑一次。

```
for(i=1;i<9;i++){int sum=0;sum+=i;}
while(i<9){i=1;sum+=i;i++}
```

4.**age > 18 && score >= 60**这种逻辑表达式用于判断真假，输出结果底层常用0或1吧。a + b * c这个就是纯数据运算。&&：并且,||或者,!逻辑非。

```
int age=20, score=75;
printf("%d\n",age>>18&&score>=60);
```

part3_函数

1.值传入只是代入数据，swap（a,b）只是在函数内把a，b的值交换了，并且变量a,b的作用域也仅存在 于函数中，对函数外没有影响。 2.要想实现值交换的目的，必须存在作用于函数外的方式。本质是把外部变量的存储位置输入函数中， 再往里面写入值（指针）